I want to demo an application for my final year project. Find attached the document. I plan on using Flutter for maximum compatibility with most main OS platforms

I want you to focus on chapter 3 mainly, coz that's where my implementation designs are. Then look for all possible libraries that we might need for the project.

So what I want you to do now is just respond with everything you understand about the project I want to do.

Hybrid-Decentralized-NetworkDigitalCommunicationnCommerceinLow.pdf

Here's what I understand about the project you want to build.

1. Core idea of the project

You want to build a proof-of-concept application for low-connectivity environments, especially places like Landmark University hostels where internet is unstable, slow, or unavailable.

The system is not just a normal chat app or file-sharing app. It is a hybrid decentralized network platform that lets nearby devices continue to communicate and exchange value even when the internet is down, and then synchronize with the internet later whenever any device regains access.

So the project is basically:

- offline-first
- peer-to-peer for local communication
- delay-tolerant
- partially decentralized locally
- centrally reconciled when internet becomes available

The name used in Chapter 3 is OffLiMU.

2. What problem the project is solving

Your project is trying to solve the dependence of modern apps on constant internet access.

Right now, most apps fail completely when the connection drops because they depend on centralized servers in real time. Your proposed system avoids that by allowing devices in the same environment to still:

- chat
- share files and media
- browse cached web content
- perform token-based local transactions
- support simple commerce workflows
- sync later through any node that temporarily has internet

So the system is designed for “graceful degradation” instead of total failure.

3. The type of system you want to demo

This is not just one app screen with one feature. It is a multi-service platform with several modules built on one shared networking backbone.

The user-facing features are:

- Chat and messaging
- File and media sharing
- Offline digital wallet / token transactions
- Local e-commerce / trading
- Offline web search / cached web browsing
- Background synchronization with server when internet returns

So what you really want to demo is a mini offline digital ecosystem.

4. The environment the system is meant for

Your case study is Landmark University student hostels.

That means the deployment assumptions are:

- many users are physically close enough for local network interactions
- internet is intermittent

- devices may move around
- some devices may temporarily have internet
- not every device is online at the same time
- the system must tolerate disconnections and delayed delivery

So the project is designed more like a campus/community offline mesh-assisted ecosystem than a classic online app.

5. The architectural philosophy

The most important design philosophy in Chapter 3 is:

- local-first
- network-agnostic app layer
- store-carry-forward networking
- eventual consistency
- opportunistic synchronization
- modular architecture

That means the app should still work from local state even if network state changes.

The UI should not depend directly on “am I online?” to function. Instead, users perform actions locally, and the system wraps those actions as events/bundles, stores them, and forwards them whenever possible.

6. Main architectural model

The system has 3 conceptual parts:

a. OffLiMU Nodes
These are ordinary participating devices like phones or laptops.

Each node can:

- run the app locally
- create and receive messages/files/transactions
- store data persistently

- forward data for others
- participate in opportunistic peer-to-peer communication

b. Gateway Nodes
A gateway is not a permanently special device.
It is just a normal node that temporarily has internet.

When internet is detected, that node can:

- upload pending data to the server
- download updates/content from the server
- synchronize payments and indexes
- act as a bridge between local offline mesh and the online world

When connectivity disappears, it goes back to normal offline node behavior.

c. Central Server
This is not for real-time control of the local network.
It exists for:

- reconciliation
- global token ledger
- scraping/fetching web content
- content aggregation
- synchronization hub functions

So your system is hybrid:
locally decentralized, globally assisted by a central authority.

7. Connectivity modes

The app is supposed to adapt to different connectivity conditions:

- Offline DTN mode:
  no internet assumed, bundles stored locally
- LAN DTN mode:
  one or more peers reachable on local network

- Gateway mode:
  internet is available, node can talk to central server
- Sync mode:
  deferred reconciliation and upload/download happens

This is important because your system behavior changes based on context, but the app experience should remain usable.

8. Internal node architecture

Chapter 3 describes a layered node architecture.

A. Application Layer
This is the user-facing module layer.

Modules include:

- messaging
- files
- wallet
- commerce
- offline search/web

These modules should:

- work with local data
- not open sockets directly
- not manage low-level networking themselves
- emit high-level intents like SendMessage, StoreFile, MakePayment, SearchContent

So this layer is isolated from networking complexity.

B. Application API Layer
This acts as the bridge between app features and the network core.

Its job is to:

- convert user intents into DTN bundles

- add metadata like source, destination, TTL, CID
- route received bundles back to the right feature module

So this is effectively your protocol adapter between app logic and transport logic.

C. DTN Core Layer
This is the real heart of the system.

It implements store-carry-forward behavior.

Main subcomponents:

1. Ingress Handler

- accepts incoming bundles from local app or remote peers
- validates structure
- checks signature
- checks TTL
- deduplicates
- admits valid bundles to storage

2. Bundle Store

- persistent storage for pending/delivered/acknowledged bundles
- survives app restarts and reboots
- acts as local DTN queue

3. Contact Manager

- discovers nearby peers
- tracks availability and contact info
- stores encounter history

4. Routing Policy Engine

- scores which peer is best to forward which bundle to
- supports routing strategies like PRoPHET or MaxProp

- separated from app logic

5. Forwarder

- checks pending bundles
- checks available contacts
- chooses what to send
- performs negotiation
- triggers transmission
- coordinates large file transfer via data layer

This means your app is not simple socket messaging. It is a proper delay-tolerant forwarding system.

D. Data Layer
This handles heavy content like:

- images
- documents
- media
- web snapshots

Instead of stuffing large binaries directly into bundles, the design uses content-addressed storage.

Main idea:

- file is stored in IPFS-like structure
- file gets chunked
- file gets a CID
- lightweight DTN bundle references the CID
- actual heavy transfer happens separately and opportunistically

This is smart because it reduces routing overhead and supports resumable transfers.

E. Transport & Connectivity Layer

This is the actual movement of bytes.

It supports:

- peer-to-peer local communication over LAN / Wi-Fi / maybe ad hoc
- secure HTTP/HTTPS synchronization to server when online

So it treats the server almost like a high-quality peer rather than redesigning the whole protocol.

9. Bundle-based communication model

A huge part of your design is that everything becomes a bundle.

That includes:

- chat messages
- file references
- payment events
- search requests
- index updates
- acknowledgements
- server confirmations

A typical bundle contains things like:

- bundle ID
- app ID
- type
- source
- destination
- timestamps
- TTL
- hop count
- priority

- payload or CID
- signature

So the system is event-driven and bundle-driven.

10. File/media transfer design

Your file transfer design is not simple direct send.

The intended workflow is:

- user selects file
- app API stores file in content-addressed storage
- data layer returns CID
- a DTN bundle containing CID is created
- routing system forwards the metadata bundle
- when a peer is authorized to fetch, the data layer performs chunk/block exchange
- transfer can pause and resume across intermittent contacts

So heavy content transfer is separated from control/routing logic.

That is one of the strongest technical ideas in your Chapter 3.

11. Payment and incentive mechanism

You are not implementing real banking.

Instead, you are designing a system-defined token economy for experimentation.

Key points:

- tokens are internal only
- they have no external monetary value
- nodes can spend tokens offline
- transactions are cryptographically signed
- local wallets are append-only logs
- balance is derived from valid incoming/outgoing events

- final validation happens only at the server during reconciliation

When offline:

- a spend event is created
- signed by sender
- propagated through DTN

When a gateway reaches internet:

- spend events are uploaded
- server checks signature
- server detects double-spend
- server reconciles ordering
- valid ones are committed
- invalid/conflicting ones are rejected
- result is broadcast back as confirmation/rejection bundles

You also want incentives:

- nodes get rewarded for forwarding data
- nodes get rewarded for acting as gateways
- server issues reward events based on logs and acknowledgements

So your payment system is really:
offline provisional transactions + centralized eventual settlement.

12. E-commerce meaning in this project

The commerce part is not a full-scale marketplace like Jumia or Amazon.

It is more like:

- local product listings
- cached catalog browsing
- placing orders locally

- provisional payment using internal tokens
- later reconciliation when online

So the commerce demo would probably show:

- a buyer browsing local listings
- making an order
- paying with internal token
- merchant receiving pending confirmation
- final server-backed settlement later

13. Offline web access and search

Another major feature is offline web search and cached browsing.

This works through three actors:

a. Server

- crawls/fetches web content from the internet
- converts pages into static snapshots
- stores them
- produces metadata/index entries

b. Gateway

- downloads new snapshots/indexes from server
- injects them into local network
- spreads update bundles

c. Offline Node

- keeps local search index
- lets user search locally
- loads cached content if available
- if not available, can issue an async search request to be resolved later

So this is not live Google search.

It is more like:

- local searchable cache
- plus deferred fetch request if a gateway later gets internet

That fits your low-connectivity use case well.

14. Acknowledgement and lifecycle management

Your design also includes ACK bundles.

These are important because in DTN, sending something does not mean it was delivered.

ACKs are used to:

- confirm successful delivery
- allow garbage collection
- remove redundant bundle copies
- support incentive accounting

So bundle lifecycle is something like:

- created
- stored
- forwarded
- delivered
- acknowledged
- eventually deleted

If TTL expires before acknowledgment:

- bundle is discarded

This is important because storage and congestion control matter in disconnected systems.

15. Role of the central server

The server is lightweight but essential.

It is responsible for things local nodes should not do alone:

- authoritative token ledger
- payment reconciliation
- double-spend detection
- web crawling and snapshot generation
- synchronization hub for geographically separated mesh groups
- content mirroring/pinning

So although the system is “decentralized,” it is not fully trustless or fully serverless.

It is better described as:
hybrid decentralized with centralized reconciliation and content aggregation.

16. Planned implementation style in the document

The document originally proposes Go-based implementation pieces like:

- Go backend/services
- WebView UI
- BoltDB
- mDNS/Zeroconf
- raw TCP
- IPFS/Kubo
- crypto libraries
- server scraper

But now you want to use Flutter for maximum compatibility.

That means, from my understanding, you want to preserve the Chapter 3 architecture while changing the implementation technology for the client app.

So in Flutter terms, your plan is likely to keep the architecture but rebuild the frontend/client around Flutter instead of the original WebView/Go-heavy prototype design.

17. How Flutter fits your project

Flutter makes sense for your goal because you want broad platform support.

This likely means you want one codebase targeting:

- Android
- Windows
- possibly Linux
- possibly macOS
- maybe iOS too, though local networking restrictions may complicate some features

But using Flutter does not remove the architectural challenges.
You still need solutions for:

- local peer discovery
- background sync
- local database
- file storage
- cryptographic signing
- content addressing
- LAN/Wi-Fi/Bluetooth communication
- server sync APIs

So Flutter can handle the app/UI/shared logic very well, but some lower-level networking or platform-specific functions may still require:

- plugins
- platform channels
- native code bridges
- companion services in Go/Rust/Kotlin/Swift if needed

18. What the demo likely needs to prove

From Chapter 3, I think the demo should prove these ideas:

1. A node can work offline

- app opens
- local state exists
- user can use features without internet

2. Two nearby nodes can communicate directly

- send message
- send file
- receive without internet

3. Data is persistent

- after app restart, pending messages/files still exist

4. Gateway behavior works

- when one device gets internet, it syncs pending data

5. Payment flow works

- offline spend event created
- later reconciled online
- confirmation or rejection comes back

6. Offline search works

- local cached content can be searched and opened
- missing content can be requested asynchronously

7. ACK lifecycle works

- delivered bundles become acknowledged
- storage queue shrinks appropriately

19. What is intentionally out of scope

Your project is a proof-of-concept, so several things are explicitly not included:

- full blockchain integration
- real money integration
- banking infrastructure
- large-scale deployment
- specialized hardware
- global-scale routing
- full adversarial security model
- full sustainability/governance model
- energy/battery optimization modeling
- complex distributed compute execution
- full iOS/Linux support in the document's original scope

So the project is about feasibility and architecture, not production-grade deployment.

20. Evaluation plan in Chapter 3

You also plan to evaluate the system using simulation and prototype deployment.

Simulation:

- use The ONE simulator
- model mobile nodes, gateways, server node
- compare routing protocols like:
  - PRoPHET
  - MaxProp
  - Epidemic

Metrics:

- delivery ratio
- average latency
- overhead ratio
- payment resolution time

- buffer occupancy

The simulation abstracts things like:

- crypto cost
- IPFS internals
- battery drain
- UI latency

So the evaluation is focused mostly on network behavior and DTN performance.

21. The kind of app architecture you are really building

In simple terms, your project is basically this:

A Flutter-based offline-first campus app platform where nearby devices can exchange messages, files, transaction events, and cached content directly, while any internet-connected device temporarily acts as a gateway to synchronize with a central server for reconciliation and broader content access.

22. My understanding of the most important technical pillars

If I boil the whole project down, these are the pillars:

- Flutter app for multi-platform UI
- Local persistent storage
- Peer discovery and peer-to-peer transport
- DTN bundle queue and forwarding logic
- Content-addressed file/media storage
- Tokenized offline transaction system
- Gateway-based sync to central server
- Offline searchable cached web content
- ACK and lifecycle management
- Server for reconciliation and web scraping

23. What I think your final demo should feel like

The strongest version of your demo would show something like this:

- Device A and Device B are offline
- A sends B a message
- A shares a file
- B receives both locally
- A pays B with internal tokens
- B sees pending receipt
- Users browse a local marketplace
- Users search already-cached web content
- Later Device C gets internet and becomes gateway
- Pending messages/logs/payments/content requests sync
- Server validates transaction
- Confirmation comes back into local network
- Missing web result becomes available later

That would clearly demonstrate your architecture.

24. One important observation

Your project is not really “just a Flutter app.”
It is a distributed systems project wrapped in an app.

The UI is only one part.
The real project is the coordination between:

- local storage
- routing
- discovery
- synchronization
- consistency
- offline event propagation
- content transfer

- eventual reconciliation

That is why Chapter 3 is the most important chapter: it defines the system behavior, not just screens.

25. In one sentence

What I understand is that you want to build a Flutter-based proof-of-concept hybrid decentralized offline-first platform called OffLiMU that enables messaging, file sharing, offline tokenized transactions, commerce, and cached web access in low-connectivity environments using DTN-style peer-to-peer communication, content-addressed storage, and opportunistic gateway synchronization with a central server.

If you want, next I can turn this understanding into either of these:

1. a clean Flutter-oriented system architecture
2. a full list of libraries/plugins you may need for each module
3. a practical MVP breakdown for what to build first
4. a mapping from your Chapter 3 design to Flutter implementation choices

Let's discuss about the clean Flutter-oriented system architecture & a full list of libraries/plugins you may need for each module

Searched the web

Searched the web

Searched the web

Absolutely — this is the right next step.

For your project, I would not build it as “just a Flutter app with screens.” I would build it as a Flutter app sitting on top of a reusable node runtime, because Chapter 3 is really describing a distributed system.

So the clean Flutter-oriented architecture should preserve your Chapter 3 design:

- app modules stay local-first
- networking is abstracted away from UI
- everything becomes events/bundles

- storage is persistent
- routing/forwarding is centralized in a node core
- gateway sync is separate from local peer communication
- transport is pluggable by platform

That is the key to making this project clean and demoable.

1. Recommended Flutter-oriented architecture

I'd structure it like this:

```text
Presentation Layer (Flutter UI)
    ↓
Feature Controllers / State Notifiers
    ↓
Application Layer (Use Cases / App API)
    ↓
Node Runtime / DTN Core
    ↓
Infrastructure Adapters
    ├── Local Database
    ├── File / Content Store
    ├── Discovery Services
    ├── P2P Transport Services
    ├── Gateway Sync Client
    ├── Background Scheduler
    └── Crypto / Key Management
```

2. Flutter architecture mapped to your Chapter 3

A. Presentation Layer

This is your Flutter UI.

Modules:

- Chat
- Files/Media
- Wallet
- Marketplace
- Offline Search

- Sync / Node status / Peer list

This layer should:

- never open sockets directly
- never talk to Wi-Fi Direct/Bluetooth directly
- only call application use-cases
- render local state from the database

Recommended packages:

- `flutter_riverpod`
- `go_router`
- `flutter_hooks` optionally
- `intl`
- `logger` optionally

B. Application Layer
This is your Flutter version of the “Application API Layer” in Chapter 3.

Responsibilities:

- convert user actions into domain commands
- create bundles/events
- pass them to the DTN core
- map incoming bundles to the correct feature module
- hide transport/network complexity from UI

Examples:

- `SendMessageUseCase`
- `CreateFileShareBundleUseCase`
- `CreateSpendEventUseCase`
- `RequestOfflineSearchUseCase`
- `TriggerGatewaySyncUseCase`

This layer should be pure Dart.

Recommended packages:

- `freezed`
- `json_serializable`
- `uuid`
- `collection`
- `equatable` or just `freezed`

C. Domain / Node Runtime / DTN Core
This is the heart of the whole app.

This is your Flutter equivalent of:

- Ingress Handler
- Bundle Store logic
- Contact Manager
- Routing Policy Engine
- Forwarder
- ACK lifecycle manager
- Sync decision logic
- Wallet reconciliation state
- Search metadata propagation

Main services I would define here:

1. `NodeRuntime`
   Starts/stops the entire node.

2. `BundleManager`
   Creates, validates, deduplicates, expires bundles.

3. `AckManager`
   Handles acknowledgements and garbage collection.

4. `ContactManager`

   Tracks peer discovery state.

5. `RoutingEngine`

   Chooses what bundle goes to what peer.

6. `Forwarder`

   Actually schedules transmissions.

7. `GatewayCoordinator`

   Detects when the node should behave like a gateway.

8. `SyncEngine`

   Uploads/downloads data to central server.

9. `WalletEngine`

   Manages tokens, spend events, confirmations, and rejections.

10. `SearchEngine`

    Maintains local search index for cached web metadata.

This layer should depend on abstract interfaces like:

- `BundleRepository`
- `PeerDiscoveryAdapter`
- `TransportAdapter`
- `ContentStore`
- `CryptoService`
- `SyncApi`
- `BackgroundScheduler`

D. Infrastructure Layer

This is where package-specific and platform-specific code lives.

Examples:

- SQLite/Drift repositories
- TCP socket transport

- mDNS discovery
- Wi-Fi Direct adapter
- Nearby adapter
- background worker adapter
- HTTP sync adapter
- secure key storage adapter

E. Platform Adapter Layer
This is critical for Flutter cross-platform support.

Do not hardcode one transport.

Instead create multiple transport/discovery adapters:

- Android:
  - Wi-Fi Direct
  - Nearby Connections
  - BLE discovery
  - LAN sockets
- Windows:
  - LAN sockets
  - mDNS/NSD discovery
- iOS/macOS:
  - LAN sockets
  - NSD/Bonjour
  - maybe Multipeer-based plugin if stable enough
- Linux:
  - LAN sockets
  - pure Dart mDNS if needed

So your app stays portable because domain logic does not know whether the message moved through:

- TCP on LAN
- Nearby API
- Wi-Fi Direct
- BLE-assisted credential exchange
- gateway HTTPS sync

3. The most important design decision

If you want maximum compatibility, your baseline transport should be:

- mDNS/NSD discovery
- TCP sockets over local LAN / hotspot

Why?

Because it is the most realistic cross-platform baseline for:

- Android
- Windows
- macOS
- iOS
- Linux

Then you add mobile-only enhancements:

- Nearby Connections
- Wi-Fi Direct
- BLE discovery

So the architecture should be:

- baseline cross-platform mode = LAN + mDNS + TCP
- enhanced Android/mobile mode = Wi-Fi Direct / Nearby / BLE

That is much cleaner than trying to make every OS use the same native P2P radio API.

4. Recommended Flutter package stack by module

Below is the cleanest package/module breakdown I'd recommend.

A. App shell, state, routing

1. `flutter_riverpod`
   Use for:

- dependency injection
- node runtime state
- feature controllers
- streams/providers for peers, bundles, messages, sync status

Why:

- very clean for complex app state
- testable
- fits layered architecture very well

Current pub.dev result seen:

- `riverpod` 3.2.1

2. `go_router`
   Use for:

- navigation between modules

3. `freezed`
   Use for:

- immutable models
- unions/sealed classes for bundle types, sync states, permission states

4. `json_serializable`
   Use for:

- serializing bundles and events

5. `uuid`

   Use for:

- bundle IDs
- message IDs
- spend event IDs

6. `logger`

   Optional, but useful for:

- node logs
- forwarding traces
- sync debugging

B. Local persistent storage

Recommended primary choice:

1. `drift`

   Use for:

- bundle store
- pending queue
- delivered/acknowledged state
- wallet transaction logs
- search index metadata
- marketplace listings/orders
- peer contact history

Why I prefer it here:

- your project is event-heavy and query-heavy
- you need structured local state
- you need transactions
- you need indexes
- append-only logs map very well to SQLite tables

Current pub.dev result seen:

- `drift` 2.32.1

Also add:

2. `sqlite3_flutter_libs`

Use for:

- SQLite native binaries on Flutter platforms

3. `path_provider`

   Use for:

- app documents dir
- app support dir
- temp/cache dirs
- download dirs

Current pub.dev result seen:

- `path_provider` 2.1.5

Alternative DB:

4. `isar`

Use if you want:

- object-style storage
- simple fast local storage
- built-in full-text search

But for your exact Chapter 3 design, I still prefer Drift.

Current pub.dev result seen:

- `isar` 3.1.0+1, with 4.0 dev prereleases

My recommendation:

- use `drift` for the main system DB

- only use `isar` if you want a more object-centric design

C. Bundle codec / local data models

1. `json_serializable`
   For MVP bundle encoding

2. `freezed`
   For model classes like:

- `Bundle`
- `AckBundle`
- `SpendEvent`
- `IndexUpdateBundle`
- `SearchRequestBundle`
- `SyncJob`
- `PeerContact`

3. `collection`
   Helpful utilities

4. `equatable`
   Optional if not using `freezed`

Optional optimization later:

5. `get_packer`

or

6. `pro_binary`

or

7. `pro_mpack`

Use only if you later want compact binary bundle encoding.

For first demo, JSON is enough and easier.

D. Connectivity awareness / gateway detection

1. `connectivity_plus`
   Use for:

- detecting available network types
- knowing when Wi-Fi/mobile/ethernet exists
- deciding when to attempt gateway sync

Important:
it does not guarantee internet access.
Use it only as signal, not proof.

Current pub.dev result seen:

- `connectivity_plus` 7.1.0

E. Peer discovery

Recommended primary choice:

1. `nsd`
   Use for:

- service discovery and registration
- Bonjour / DNS-SD / mDNS on supported platforms

This is very aligned with your Contact Manager idea.

Current pub.dev result seen:

- `nsd` 4.1.0
- supports Android, iOS, macOS, Windows

Good fallback / pure Dart option:
2. `mdns_dart`
Use for:

- pure-Dart mDNS discovery/advertisement
- more control
- cross-platform networking without native dependency

Current pub.dev result seen:

- `mdns_dart` 2.2.1

Publishing-only mobile option:

3. `mdns_responder`

Use for:

- mDNS service advertisement on Android/iOS
- if you need more direct service publishing features

Current pub.dev result seen:

- `mdns_responder` 0.0.2

My recommendation:

- use `nsd` first
- keep `mdns_dart` as fallback or advanced option

F. Peer-to-peer transport

Baseline cross-platform transport:

1. `dart:io`

   Use for:

- TCP server sockets
- TCP client sockets
- UDP if needed
- direct peer bundle transmission over LAN

This should be your default transport implementation.

No plugin required.

Why:

- stable
- fast
- available across Flutter native targets

- ideal for Windows + Android demo

Mobile-enhanced P2P options:

2. `flutter_p2p_connection`
   Use for:

- Android Wi-Fi Direct group-based communication
- BLE-assisted discovery/credential exchange
- text/file transfer support

Current pub.dev result seen:

- `flutter_p2p_connection` 3.0.x
- Android-focused

3. `wifi_direct_plugin`
   Use for:

- cross-platform-ish Wi-Fi Direct / Multipeer-style abstraction
- text and file transfer
- Android + iOS listed

Current pub.dev result seen:

- `wifi_direct_plugin` 0.1.7

4. `flutter_nearby_connections_plus`
   Use for:

- Nearby Connections style mobile discovery and communication
- Android + iOS
- message, stream, file/resource transport

Current pub.dev result seen:

- `flutter_nearby_connections_plus` 1.2.2

BLE-specific:

5. `flutter_blue_plus`

Use for:

- BLE scanning / BLE communication
- best as discovery or credential exchange helper
- not ideal for large file transfer itself

Current pub.dev result seen:

- `flutter_blue_plus` 2.2.1

My recommendation:

- core demo transport: `dart:io` TCP sockets
- peer discovery: `nsd`
- Android enhancement: `flutter_p2p_connection` or `flutter_nearby_connections_plus`
- BLE should be auxiliary, not your main transport for payloads

G. Content-addressed storage / IPFS layer

This part is the trickiest in Flutter.

You have 3 realistic options.

Option 1 — best for clean academic alignment:

1. `dart_ipfs`

Use for:

- pure Dart IPFS node
- offline content-addressed storage
- Bitswap/P2P features
- Flutter/mobile/web support claims

Current pub.dev result seen:

- `dart_ipfs` 1.9.4

Pros:

- closest to Chapter 3 vision
- pure Dart
- avoids external Kubo dependency

Cons:

- you must test it early
- likely more integration risk than standard app packages

Option 2 — practical hybrid:
2. local custom content-addressed store
Use:

- compute SHA-256 hash
- split file into chunks
- store chunks locally using filesystem
- store CID-to-metadata in Drift
- transfer chunks using your own forwarder

This is not a package, but honestly for final year demo it may be the safest.
It still satisfies the Chapter 3 concept:

- content addressing
- chunking
- dedup
- resumable transfer

Option 3 — desktop/server sidecar:
3. external Kubo node + HTTP RPC
Use Flutter only as client via:

- `dio`

  or

- `ipfs_client_flutter`

Current pub.dev result seen:

- `ipfs_client_flutter` 1.0.7
- but it is older and RPC-only

My recommendation:

- for MVP demo: custom local content-addressed store first
- for strong IPFS alignment: test `dart_ipfs` early
- use external Kubo only if you're okay shipping a sidecar process on desktop

H. Crypto / identity / signatures

You need:

- node keypair generation
- signatures on bundles
- SHA hashing for bundle/file IDs
- optionally encryption later

Recommended:

1. `cryptography`
   or maintained equivalent in the same family

From search, the currently active maintained option surfaced as:

2. `cryptography_plus`

Current pub.dev result seen:

- `cryptography_plus` 3.0.0

Use for:

- Ed25519 signatures
- SHA-256 hashing
- X25519 if ever needed

- symmetric encryption if you later add it

Also use:

3. `flutter_secure_storage`

or another secure keystore wrapper

Use for:

- storing private keys securely on device

I'd use:

- Ed25519 for node identity + signing
- SHA-256 for content addressing
- secure storage for private key material

I. Gateway sync / central server communication

1. `dio`

   Use for:

- sync upload/download
- retries
- interceptors
- multipart upload
- timeout handling
- background-friendly HTTP client

Current pub.dev result seen:

- `dio` 5.9.2

2. `connectivity_plus`

   Use as a signal for when sync may be attempted

3. `workmanager`

   Use for:

- mobile deferred sync tasks

Current pub.dev result seen:

- `workmanager` 0.9.0+3

Alternative:

4. `native_workmanager`

Use if:

- you want lower-overhead native background tasks
- you're okay with a newer package

Current pub.dev result seen:

- `native_workmanager` 1.1.0

My recommendation:

- use `dio` for sync client
- use `workmanager` first
- only consider `native_workmanager` if background execution becomes a pain point

J. Background execution

For Android/iOS:

1. `workmanager`

   Best for:

- periodic sync
- retrying pending uploads
- refreshing search metadata
- cleaning expired bundles

Current pub.dev result seen:

- `workmanager` 0.9.0+3

For desktop:

- you usually don't need a plugin

- run scheduler inside app runtime
- use isolates/timers while app is open
- optionally register startup behavior natively later

So define a `BackgroundScheduler` interface:

- Android/iOS implementation = `workmanager`
- desktop implementation = Dart timer/isolate scheduler

K. Files and media

1. `file_picker`
   Use for:

- choosing documents/media across Android, iOS, Windows, Linux, macOS, web

Current pub.dev result seen:

- `file_picker` 11.0.1

2. `image_picker`
   Optional for:

- quick media capture on mobile

3. `path_provider`
   Use for:

- storing transferred files and snapshots

4. `mime`
   Use for:

- content type detection

Optional preview packages:

5. `video_player`
6. `photo_view`
7. `open_filex` or similar open-file helper

Core recommendation:

- `file_picker`
- `path_provider`
- `mime`

L. Offline web browsing and cached HTML rendering

Primary options:

1. `webview_flutter`
   Use for:

- rendering saved HTML snapshots
- displaying cached web pages locally

Current pub.dev result seen:

- `webview_flutter` 4.13.1

2. `flutter_inappwebview`
   Use if:

- you want more control
- local content handling is more complex
- you need headless webview / advanced interception

Current pub.dev result seen:

- `flutter_inappwebview` 6.1.5

3. `webview_flutter_plus`
   Use if:

- you want easier local HTML/CSS/JS loading via local server approach

Current pub.dev result seen:

- `webview_flutter_plus` 0.4.20

My recommendation:

- start with `webview_flutter`
- move to `flutter_inappwebview` only if local snapshot rendering becomes limiting

M. Permissions

1. `permission_handler`
   Use for:

- Bluetooth
- location
- storage/media
- notifications
- nearby devices
- network-related permissions where needed

Current pub.dev result seen:

- `permission_handler` 12.0.1

Important note:

- it supports Android, iOS, web, Windows
- not full Linux/macOS support in the current package score page

So for desktop, you may need:

- platform-specific entitlements/config
- not just runtime permission package calls

N. Notifications

You'll likely need local notifications for:

- background sync completed
- file transfer finished
- payment confirmed/rejected

- peer discovered
- request pending

Recommended:

1. `flutter_local_notifications`

Very useful for:

- background feedback
- gateway status alerts
- sync confirmation UX

O. Search index module

For your Chapter 3 search engine:

- metadata entries
- keyword tokenization
- local lookup
- async request queue

Best implementation:

- store metadata and token maps in Drift/SQLite
- optionally use SQLite FTS5
- or simple inverted index tables

No special search package is required at first.

So:

- `drift`
- `sqlite3_flutter_libs`

are enough.

P. Testing / debugging / dev tooling

Recommended:

- `flutter_test`
- `integration_test`
- `mocktail`
- `build_runner`
- `freezed`
- `json_serializable`
- `very_good_analysis` or `flutter_lints`

5. The clean interface-first architecture I recommend

Do this with abstract interfaces.

Example service interfaces:

```text
abstract class DiscoveryAdapter
abstract class TransportAdapter
abstract class ContentStore
abstract class BundleRepository
abstract class WalletRepository
abstract class SyncApi
abstract class BackgroundScheduler
abstract class CryptoService
```

Then implementations:

```text
DiscoveryAdapter
├── NsdDiscoveryAdapter
├── MdnsDartDiscoveryAdapter
└── NearbyDiscoveryAdapter

TransportAdapter
├── TcpSocketTransportAdapter
├── NearbyTransportAdapter
├── WifiDirectTransportAdapter
└── BleTransportAdapter

ContentStore
```

```
├── LocalCasContentStore
├── DartIpfsContentStore
└── KuboRpcContentStore
```

That way:

- your app stays clean
- your Chapter 3 architecture remains visible
- you can swap implementations without rewriting UI

6. My recommended practical stack for your FYP

If I were building this for your demo, this is the stack I would choose first:

Core mandatory stack

- `flutter_riverpod`
- `drift`
- `sqlite3_flutter_libs`
- `path_provider`
- `dio`
- `connectivity_plus`
- `permission_handler`
- `file_picker`
- `webview_flutter`
- `uuid`
- `freezed`
- `json_serializable`
- crypto package from the `cryptography` family
- `workmanager`
- `nsd`
- `dart:io` sockets

Then mobile enhancement stack

- `flutter_p2p_connection` or `flutter_nearby_connections_plus`
- `flutter_blue_plus` only if needed for BLE discovery

Then IPFS/content-addressing choice

- safest MVP: custom local CAS
- stronger research alignment: `dart_ipfs`

7. What I would not do for the first version

I would not start by combining all of these at once:

- Wi-Fi Direct
- Nearby Connections
- BLE transport
- full IPFS node
- background workers
- marketplace
- wallet
- search
- sync

That is too much risk at once.

Instead:

Version 1:

- Flutter UI
- Drift DB
- bundle engine
- LAN discovery with `nsd`
- TCP socket transport
- chat
- file transfer

- ACK lifecycle
- simple server sync

Version 2:

- wallet + provisional spend events
- commerce listings/orders
- offline search metadata

Version 3:

- Android Wi-Fi Direct / Nearby / BLE enhancement
- optional `dart_ipfs`

That still matches your dissertation, but gives you something demoable sooner.

8. Suggested Flutter folder structure

```text
lib/
├── app/
│   ├── app.dart
│   ├── router.dart
│   └── providers.dart
├── core/
│   ├── constants/
│   ├── errors/
│   ├── utils/
│   ├── crypto/
│   └── logging/
├── domain/
│   ├── entities/
│   │   ├── bundle.dart
│   │   ├── ack_bundle.dart
│   │   ├── spend_event.dart
│   │   ├── peer_contact.dart
│   │   └── index_entry.dart
│   ├── repositories/
│   ├── services/
│   └── usecases/
├── infrastructure/
│   ├── db/
│   ├── discovery/
│   ├── transport/
```

```
│   ├── storage/
│   ├── sync/
│   ├── background/
│   └── crypto/
├── node_runtime/
│   ├── node_runtime.dart
│   ├── bundle_manager.dart
│   ├── ack_manager.dart
│   ├── contact_manager.dart
│   ├── routing_engine.dart
│   ├── forwarder.dart
│   ├── gateway_coordinator.dart
│   └── sync_engine.dart
└── features/
    ├── chat/
    ├── files/
    ├── wallet/
    ├── marketplace/
    ├── search/
    └── settings/
```

9. Final recommendation in one line

For a clean Flutter implementation of Chapter 3, build an interface-driven offline-first node runtime in pure Dart, use Flutter only for UI and orchestration, use Drift for persistence, mDNS/NSD + TCP as the baseline cross-platform transport, mobile-specific P2P plugins as optional adapters, and keep IPFS pluggable rather than hardwired from day one.

If you want, next I can do either of these:

1. give you a final recommended package list as a clean table of:
    - package
    - purpose
    - mandatory/optional
    - platforms
    - my recommendation
2. design the exact Flutter class architecture for:
    - NodeRuntime
    - Bundle model

- repositories
- transport adapters
- sync engine

3. propose the best MVP feature order for implementation over, say, 4–8 weeks

> Search Grounding Details

continue from where you stopped

Stopped by user